Module 5: Database Security

Database Systems I

In This Module

- What are the threats to database security?
- What control measures can be taken to provide security of data in relational databases?
- What is SQL injection and how can it be used to compromise database security?

Learning Outcomes

After successfully completing this module you should be able to explain key security concepts related to database control measures and SQL injection.

- Explain at a high-level the four database control measures: access control, inference control, flow control and data encryption.
- Describe at a high level how the discretionary access control mechanism works.
- Given a set of commands for granting and revoking privileges, determine which users would have what level of access to which resources.
- Describe at a high level how the mandatory access control mechanism works.
- Given a set of users and resources with their classification rankings, apply the Bell-LaPadula model to determine which users would have what level of access to which resources.
- Describe at a high level how the role-based access control mechanism works.
- Explain SQL injection and the risks associated with it

Intro to Database Security Issues

Access Control Mechanisms

Other Control Measures

SQL Injection

Why Database Security

Legal or policy reasons

- E.g. government polices that intend to protect the access of individuals' data (medical records, financial rating, etc)

Ethical reasons

- E.g. accessing employee salary packages, student grades …

Technical reasons

- E.g. protection from malware, performance overloading, …

Threats to Database Security

Loss of confidentiality

- Unauthorized disclosure of confidential information

Loss of integrity

- Improper modification of information

Loss of availability

- Legitimate user cannot access data objects

(Known as C.I.A principle)

Privacy and Security

Security

- Database security concerns how to control the access of data
  - Availability for use (with integrity) and permitted access

Privacy

- Privacy concerns how the data can be used
- It's about the ability of individuals to control the terms under which their sensitive data is acquired and used
  - Preventing storage of sensitive data
  - Ensuring appropriate/authorized use of sensitive data

Security and privacy are closely related but different

- Security is a required building block for privacy
- Privacy is about both directly accessible information and inferred information

Sensitive Data

Patient data is private → Patient data has insights for scientific research on biomedicine, drug innovation, public health …

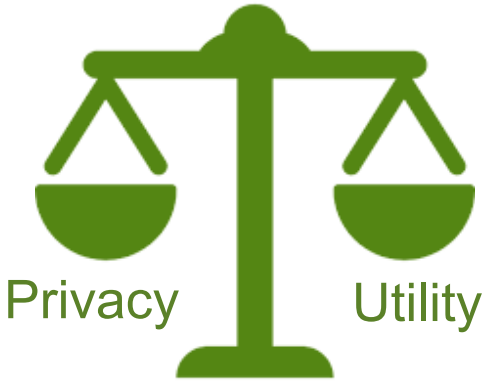
How can access to patient's data be (mis)used?

CCTV cameras can record private conversations from general public → Audio data can contain critical information on criminal, and terrorist activity

How can access to data on people's movements, behaviours and social habits be (mis)used?

Utility and Privacy Tradeoff

Typically a tradeoff between accessibility and security



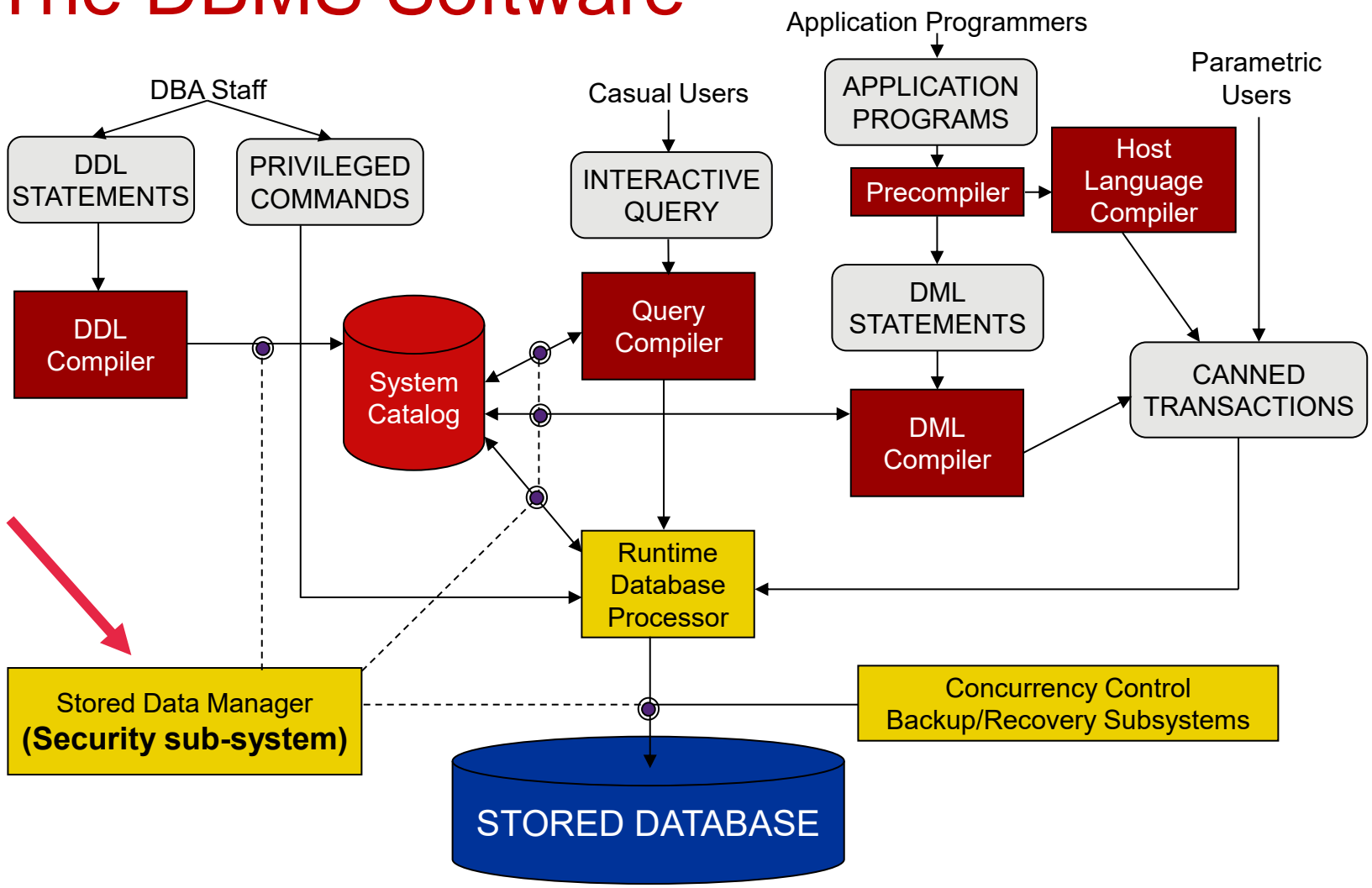
Accessibility

- Protect all sensitive data while making available as much nonsensitive data as possible

Security

- Ensuring data kept safe from corruption and access suitably controlled

The DBMS Software



Database Control Measures

Access control

- Provide access only to users who have the right access authority.

Inference control

- Must ensure information about individuals cannot be accessed (applies to statistical databases)

Flow control

- Prevents information from flowing to unauthorized users

Data encryption

- Used to protect sensitive transmitted data

Intro to Database Security Issues

Access Control Mechanisms

Other Control Measures

SQL Injection

Access Control Mechanisms

Database access control is a method of allowing access to sensitive data only to users who are authorised to access such data and to restrict access to unauthorized persons. It includes two main components

- Authentication: verifying the identity of a person who is accessing your database
- Authorization: determining whether a user should be allowed to execute the transaction he's attempting.

Basic DBMS Authentication Security Functions

User Accounts

- User must log in using assigned username and password

Login session

- Sequence of database operations by a certain user
- Recorded in **system log** (also used for recovery and other control functions, not just security)

Database audit

- Reviewing log to examine all accesses and operations applied during a certain time period

DB Audit

username	password	sessionStart	sessionEnd	sessionID
Jake	J15a	01.03.06.11.03	01.03.06.11.12	78
Nelly	Mb67	21.07.09.13.34	21.07.09.13.49	209
Victor	Y87g	…		

sessionID	object	beforeValue	afterValue
78	CDB.ORDER.Amt	2590	2700
209	CDB.CUST.cname	A. Kern	Anika Kern
…			

DB Audit

event_time	user_host	thread_id	server_id	argument
17-05-2020 8:35:43 am	StockManager @ localhost	2129	1	DELETE FROM SummerStyles.Deliveries WHERE id = 204472
17-05-2020 8:38:32 am	StockManager @ localhost	2128	0	UPDATE SummerStyles.CurrentStock SET quantity = quantity + 8 WHERE stockID = 1263
17-05-2020 8:35:05 am	CustomerService @ localhost	2129	2	SELECT fName, lName, max(wage) FROM SummerStyles.Staff

A database log can be used to identify potential cases of privacy breaches or unauthorised view or modification of data.

Access Control Mechanisms

Authorization: determining whether a user should be allowed to execute the transaction he's attempting.

Discretionary Access Control (DAC)

- Used to grant privileges to users

Mandatory Access Control (MAC)

- Classify data and users into various security classes
- Implement security policy

Role-based Access Control (RBAC)

- Users can be assigned with roles, and then DAC or MAC can be applied

Discretionary Access Control

Based on Granting and Revoking privileges

```
CREATE USER <username> IDENTIFIED BY "<password>";

GRANT <privilege> TO <user>;
```

```
GRANT privilegeName
ON        objectName
TO        {userName |PUBLIC |roleName}
[WITH GRANT OPTION];
```

```
REVOKE privilegeName
ON         objectName
FROM     {userName |PUBLIC |roleName}
```

Two Level of Privileges

Account level: privileges specified for each account, independent of relations in the account

- Create schema, create table, create view; alter/drop tables; modify (insert, delete, update tuples) and select privileges

Relation level: privileges for each relation or view

- Can be specified at attribute level too
- Select (retrieval or read) privilege
- Modification (insert, deleted and update) privilege
- References privilege

Example: CREATE SCHEMA

```
CREATE SCHEMA Factory AUTHORIZATION A1

   CREATE TABLE Product
      (color VARCHAR2(10) PRIMARY KEY, quantity NUMBER)

   CREATE VIEW RedProduct AS
      SELECT color, quantity FROM Product WHERE color = 'RED’

   GRANT select ON RedProduct TO hr;
```

The above statement

- creates a schema named *Factory*
- Specifies the owner as *A1*
- creates the table *Product;*
- creates the view *RedProduct;*
- grants the SELECT object privilege on *RedProduct* to another user *hr*.

Access Matrix Model

Each relation R is assigned an owner account

Owners can grant (select, modification and references) privileges to other users on any owned relation

	Jake	Aiden	Nelly	Sham
Customer	Read	READ	Modify	X
CustOrder	X	X	Read	X
CustOoder.amt	X	READ	Read	X
Supp.status	X	READ	Modify	Reference
Supp.address	X	READ	Modify	Modify

Each M(i, j) represents the type of privileges (read, modify, reference) that a user i has on an object j

The REFERENCES privilege type grants permission to create a foreign key reference to the specified table.

Privilege Revocation/Propagation

Revoking of Privileges

- Useful for granting a privilege temporarily
- REVOKE command used to cancel a privilege

Propagation of privileges using the GRANT OPTION

- If GRANT OPTION is given to B, B can grant privilege to other accounts without knowledge of owner
- DBMS must keep track of how privileges were granted if DBMS allows propagation (*a chain of dependency*)
- This can be very complex

REVOKE [RESTRICT|CASCADE]

- If you use CASCADE, it will revoke the privilege and any dependent privileges as a result of your grant
- If you use RESTRICT, and the privilege has been passed, the execution returns error

Specifying Privileges Through Views

If the owner A of a relation R wants another account B to be able to retrieve only some fields of R, then A can create a view V of R

Employee [ssn, name, dob, address, sex, salary, mgrSSN, dNum]

Suppose that A1 wants to give A3 a limited capability to SELECT from the EMPLOYEE relation and wants to allow A3 to retrieve only the Name, Bdate, and Address attributes and only for the tuples with Dno = 5.

```
CREATE VIEW A3_EMPLOYEE AS
    SELECT Name, Bdate, Address
    FROM EMPLOYEE
    WHERE Dno = 5
```

```
GRANT SELECT ON A3_EMPLOYEE TO A3
```

Question 1: Discretionary Access Control

Which of the following statement is used to remove the privilege from a user?

A. Remove update on department from Amir
B. Revoke update on employee from Amir
C. Delete select on department from Raj
D. Grant update on employee from Amir

Question 1: Discretionary Access Control

Which of the following statement is used to remove the privilege from a user?

- A. Remove update on department from Amir
- B. Revoke update on employee from Amir — Correct
- C. Delete select on department from Raj
- D. Grant update on employee from Amir

Question 2: Discretionary Access Control

If u1 provides authorization to u2 which in turn u2 gives to u3 which of the following is correct ?

A. If u1 revokes authorization from u3 then u2's authorization is revoked
B. If u1revokes authorization from u2 then u3's authorization is also revoked
C. If u2's authorization is revoked then then u1's authorization is also automatically revoked.

Question 2: Discretionary Access Control

If u1 provides authorization to u2 which in turn u2 gives to u3 which of the following is correct ?

- ✗ A. If u1 revokes authorization from u3 then u2's authorization is revoked
- ✓ B. If u1 revokes authorization from u2 then u3's authorization is also revoked — Correct
- ✗ C. If u2's authorization is revoked then then u1's authorization is also automatically revoked.

A user has an authorization if and only if there is a path from the root of the authorization graph down to the node representing the user.

u1
↓
u2
↓
u3

Access Control Mechanisms

Authorization: determining whether a user should be allowed to execute the transaction he's attempting.

Discretionary Access Control (DAC)

- Used to grant privileges to users

Mandatory Access Control (MAC)

- Classify data and users into various security classes
- Implement security policy

Role-based Access Control (RBAC)

- Users can be assigned with roles, and then DAC or MAC can be applied

Mandatory Access Control

Additional security policy that classifies data and users based on security classes

Used for multilevel security

Data and users are associated with security classes

- Typical security classes
  - Top secret (TS)
  - Secret (S)
  - Confidential (C)
  - Unclassified (U)

User: subject

Data (table/tuple/attribute): object

DBMS Accounts	
Database User	Access Level
Cody	U
Jane	C
Tanya	S
Jack	TS

(a) EMPLOYEE

TC: Tuple Classification

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Brown C	80000 S	Good C	S

The Bell-LaPadula Model

Simple security property

- A subject can't read information from an object that has a higher sensitivity label than the subject
- Also known as: *no read up*, or *NRU*
- Example: Database user Cody with access level Unclassified cannot view Smith's salary which requires Confidential level access

DBMS Accounts	
Database User	Access Level
Cody	U
Jane	C
Tanya	S
Jack	TS

(a) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Brown C	80000 S	Good C	S

Top secret (TS) > Secret (S) > Confidential (C) > Unclassified (U)

The Bell-LaPadula Model

Star property

- A subject can't write information to an object that has a lower sensitivity label than the subject
- Also known as: *no write down*, or *NWD*
- This is to prevent information from flowing from higher to lower classifications
- Example: Database user Tanya with access level Secret cannot insert a new tuple with information about Smith with Confidential level access.

DBMS Accounts	
Database User	Access Level
Cody	U
Jane	C
Tanya	S
Jack	TS

(a) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Brown C	80000 S	Good C	S

Top secret (TS) > Secret (S) > Confidential (C) > Unclassified (U)

Multilevel Security Example

Example: A multilevel relation to illustrate multilevel security

a) The original EMPLOYEE tuples

b) Appearance of EMPLOYEE after filtering for classification C users

c) Appearance of EMPLOYEE after filtering for classification U users

d) Polyinstantiation of the Smith tuple (a database to maintain multiple records with the same key, in order to prevent inference attacks)

(a) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Brown C	80000 S	Good C	S

(b) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	NULL C	C
Brown C	NULL C	Good C	C

(c) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	NULL U	NULL U	U

(d) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Smith U	40000 C	Excellent C	C
Brown C	80000 S	Good C	S

Top secret (TS) > Secret (S) > Confidential (C) > Unclassified (U)

Example of Inference Attacks

Suppose a user with security clearance C tries to update the value of JobPerformance for Smith from NULL to 'Excellent' in (b)

```
UPDATE EMPLOYEE
SET JobPerformance = 'Excellent'
WHERE Name = 'Smith';
```

System should not reject it otherwise user could *infer* that some nonnull value exists

This is an example of inferring information through a *covert channel*

Solution is polyinstantiation

(a) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Brown C	80000 S	Good C	S

(b) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	NULL C	C
Brown C	NULL C	Good C	C

(d) EMPLOYEE

Name	Salary	JobPerformance	TC
Smith U	40000 C	Fair S	S
Smith U	40000 C	Excellent C	C
Brown C	80000 S	Good C	S

Top secret (TS) > Secret (S) > Confidential (C) > Unclassified (U)

Discretionary vs Mandatory Access Control

Discretionary policies have a high degree of flexibility

- Do not impose control on how information is propagated

Mandatory policies ensure high degree of protection

- Rigid but prevent illegal information flow

Access Control Mechanisms

Authorization: determining whether a user should be allowed to execute the transaction he's attempting.

Discretionary Access Control (DAC)

- Used to grant privileges to users

Mandatory Access Control (MAC)

- Classify data and users into various security classes
- Implement security policy

Role-based Access Control (RBAC)

- Users can be assigned with roles, and then DAC or MAC can be applied

Role-Based Access Control

Permissions associated with organizational roles

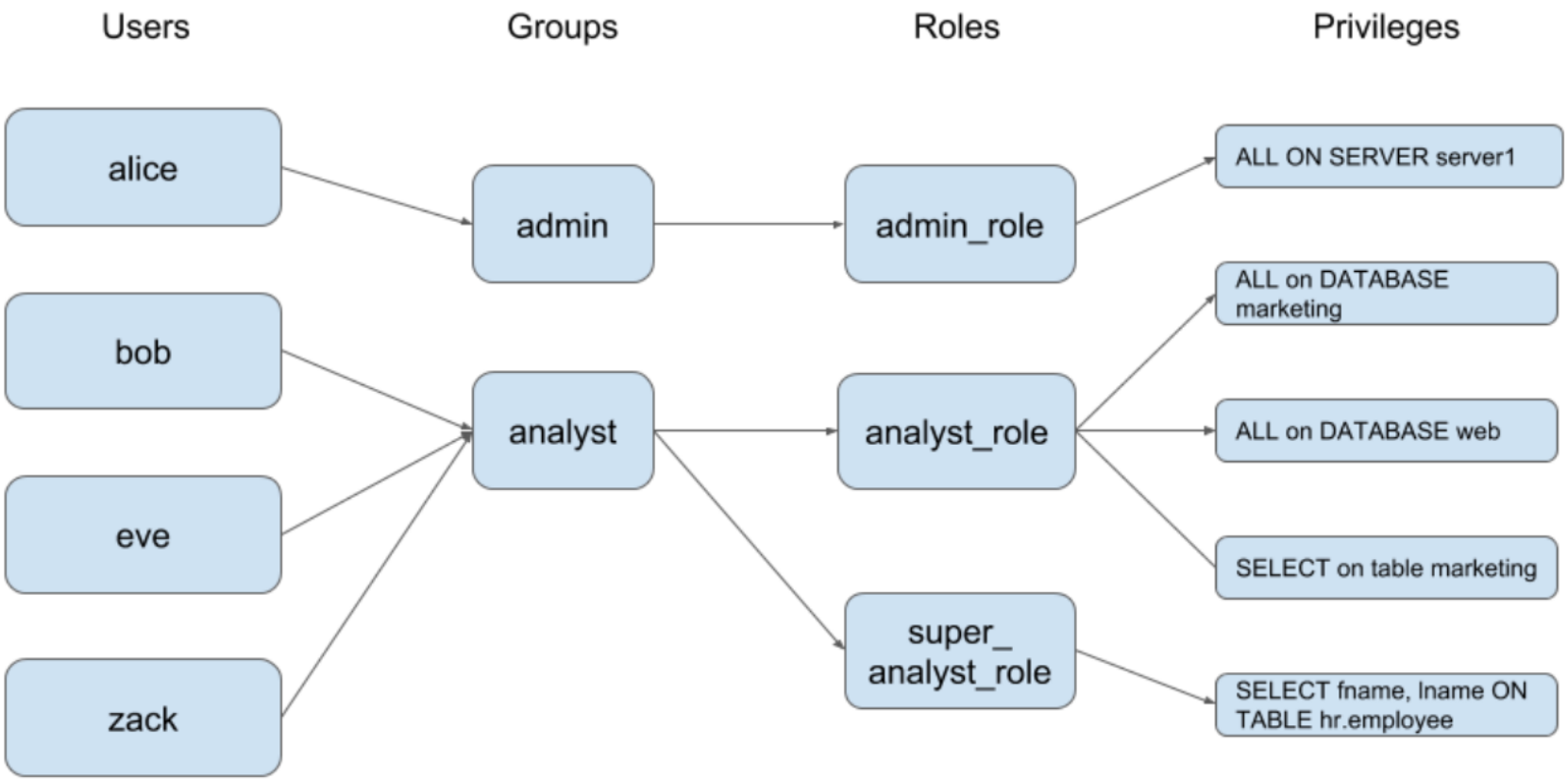
- Users are assigned to appropriate roles

```
GRANT ROLE full-time TO emp_typ1;
GRANT ROLE intern TO emp_typ2;
```

Roles can be used with traditional DAC and MAC methods

This model has several desirable features, such as flexibility and better support for administration making it an attractive candidate for web-based applications.

An Example of Role-Based Access Control



SQL Support for Role-Based Access Control

Create/drop roles

```
CREATE ROLE manager;
DROP ROLE manager;
```

Grant/revoke privileges

```
GRANT privilegeName
ON        objectName
TO        {userName | PUBLIC |roleName}
[WITH GRANT OPTION];

REVOKE privilegeName
ON        objectName
FROM      {userName | PUBLIC |roleName}
```

An Exercise

Assume that there are 4 users: A1, A2, A3, A4

- DBA issues: GRANT CREATETAB TO A1;
- A1 creates two base relations: Employee and Department
- A1 gives following privileges:
  - GRANT INSERT, DELETE ON Employee, Department TO A2;
  - GRANT SELECT ON Employee, Department TO A3 WITH GRANT OPTIONS;

Q1: Can the followings be executed by user A3?

- GRANT SELECT ON Employee TO A4?

Q2: If A1 issues REVOKE SELECT ON Employee FROM A3, does A4 still have SELECT privilege on Employee?

Q3: What type of Access Control is being used: Discretionary, Mandatory or Role Based?

An Exercise

Assume that there are 4 users: A1, A2, A3, A4

- DBA issues: GRANT CREATETAB TO A1;
- A1 creates two base relations: Employee and Department
- A1 gives following privileges:
  - GRANT INSERT, DELETE ON Employee, Department TO A2;
  - GRANT SELECT ON Employee, Department TO A3 WITH GRANT OPTIONS;

Q1: Can the followings be executed by user A3? GRANT SELECT ON Employee TO A4
A1: Yes, A3 has granting option of SELECT on the EMPLOYEE relation.

Q2: If A1 issues REVOKE SELECT ON EMPLOYEE FROM A3, does A4 still have SELECT privilege on EMPLOYEE?
A2: No, The privileges of A4 will automatically be revoked due to propagation.

Q3: What type of Access Control is being used: Discretionary, Mandatory or Role Based?
A3: Discretionary access control is being used.

Intro to Database Security Issues

Access Control Mechanisms

Other Control Measures

SQL Injection

Database Control Measures

Access control

- Provide access only to users who have the right access authority.

Inference control

- Must ensure information about individuals cannot be accessed (applies to statistical databases)

Flow control

- Prevents information from flowing to unauthorized users

Data encryption

- Used to protect sensitive transmitted data

Statistical databases

Statistical databases are used to provide statistics about various populations.

Statistical database users such as government statisticians or market research firms are allowed to retrieve statistical information about a population but are not authorized to access detailed confidential information.

Only **statistical queries** that involve statistical aggregate functions such as COUNT , SUM , MIN , MAX, AVERAGE , and STANDARD DEVIATION are allowed

Q1: **SELECT COUNT** (*) **FROM** PERSON
WHERE <condition>;
Q2: **SELECT AVG** (Income) **FROM** PERSON
WHERE <condition>;

Threats to Statistical Databases Example

Example of running a sequence of queries to identify information about an individual.

Student Grades (Redacted View)

Grade	Course	Year	DegreeName	Gender
B+	CMPT354	2	Computer Science	F
C+	PHYS255	2	Physics	M
A	PHYS255	4	Physics	F
A-	CMPT354	2	Computer Science	M
A+	PHYS255	1	Physics	F
B	PHYS255	1	Physics	F
A+	CMPT354	3	Computer Science	M
B-	PHYS255	3	Physics	M

Threats to Statistical Databases Example

```
SELELCT AVG (Grade)
FROM      Student Grades
WHERE Course like 'CMPT354'
```

Grade	Course	Year	DegreeName	Gender
B+	CMPT354	2	Computer Science	F
C+	PHYS255	2	Physics	M
A	PHYS255	4	Physics	F
A-	CMPT354	2	Computer Science	M
A+	PHYS255	1	Physics	F
B	PHYS255	1	Physics	F
A+	CMPT354	3	Computer Science	M
B-	PHYS255	3	Physics	M

Threats to Statistical Databases Example

```
SELELCT AVG (Grade)
FROM      Student Grades
WHERE Course like 'CMPT354' AND Year = 2
```

Grade	Course	Year	DegreeName	Gender
B+	CMPT354	2	Computer Science	F
C+	PHYS255	2	Physics	M
A	PHYS255	4	Physics	F
A-	CMPT354	2	Computer Science	M
A+	PHYS255	1	Physics	F
B	PHYS255	1	Physics	F
A+	CMPT354	3	Computer Science	M
B-	PHYS255	3	Physics	M

Threats to Statistical Databases Example

```
SELELCT AVG (Grade)
FROM      Student Grades
WHERE Course like 'CMPT354' AND Year = 2 AND Gender = 'F'
```

Grade	Course	Year	DegreeName	Gender
B+	CMPT354	2	Computer Science	F
C+	PHYS255	2	Physics	M
A	PHYS255	4	Physics	F
A-	CMPT354	2	Computer Science	M
A+	PHYS255	1	Physics	F
B	PHYS255	1	Physics	F
A+	CMPT354	3	Computer Science	M
B-	PHYS255	3	Physics	M

Inference Control

- Preventing the inference of individual information
  - Provide minimum threshold on number of tuples (k-anonymity)

```
SELELCT AVG (Grade)
FROM      Student Grades
WHERE     Where Course like 'CMPT354' AND Year = 2 AND Gender = 'F'
```

- Prohibit sequences of queries that refer to the same population of tuples

- Introduce slight noise or inaccuracy (check out differential privacy)

Flow Control

Flow control

- Regulates the distribution or flow of information among accessible objects
- Verifies information contained in some objects does not flow explicitly or implicitly into less protected objects

Flow policy

- Specifies channels along which information is allowed to move
- Simple form: confidential (C) and nonconfidential (N) … prohibit flow from C to N

Example:

- Income Tax Computing Service is allowed to retain customer address but not income and deductions data

Data Encryption

Encryption converts data into cyphertext

- Performed by applying an encryption algorithm to data using a prespecified encryption key
- Resulting data must be decrypted using a decryption key to recover original data

Data Encryption Standard (DES)

- Developed by the U.S. Government for use by the general public

Advanced Encryption Standard (AES)

- More difficult to crack

Intro to Database Security Issues

Access Control Mechanisms

Other Control Measures

SQL Injection

SQL Injection

SQL injection

- One of the most common threats to a database system.

- Attacker injects a string input through the (often web) application which changes or manipulates SQL statement to attacker's advantage

SQL Injection Methods

SQL manipulation

- Changes an SQL command in the application
- Example: adding conditions to the WHERE clause
- Typical manipulation attack occurs during database login

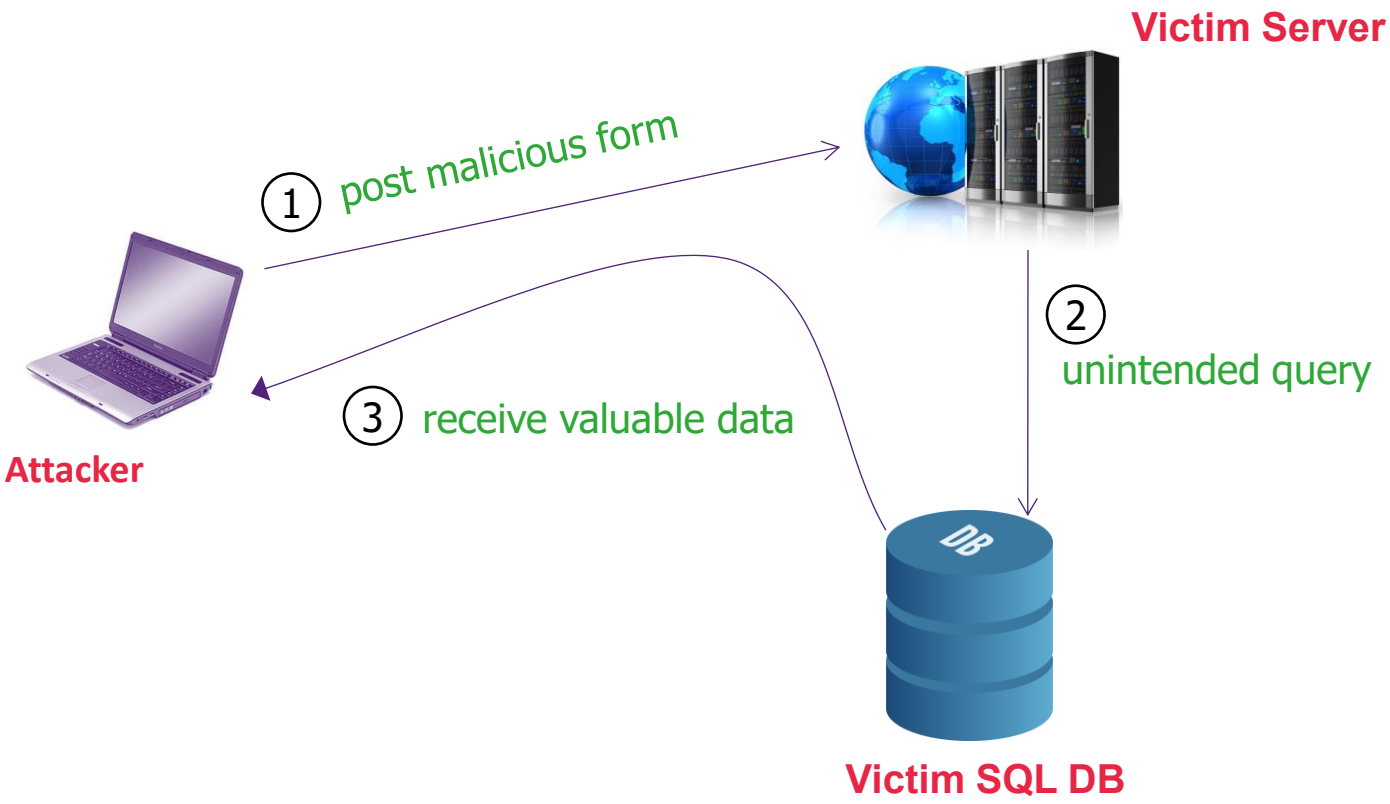
Code injection

- Add additional SQL statements or commands that are then processed

Function call injection

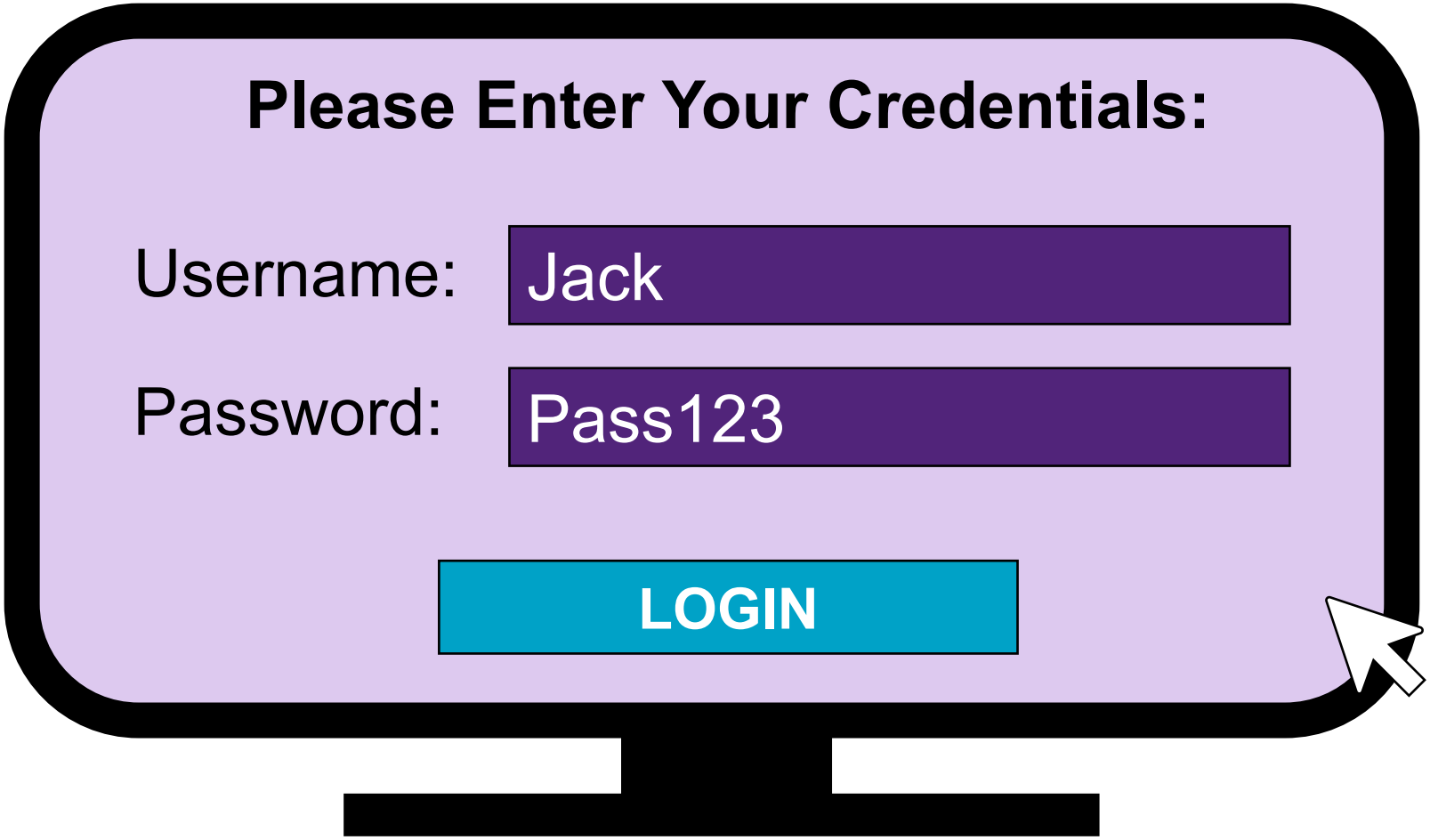
- Database or operating system function call inserted into vulnerable SQL statement to manipulate data or make a privileged system call

SQL Injection



SQL Manipulation Example

Regular behavior



SQL Manipulation Example

Regular behavior

DBMS

```
SELECT *
FROM Users
WHERE Username = ‘Jack’ AND Password = ‘Pass123’;
runQuery(query);
```

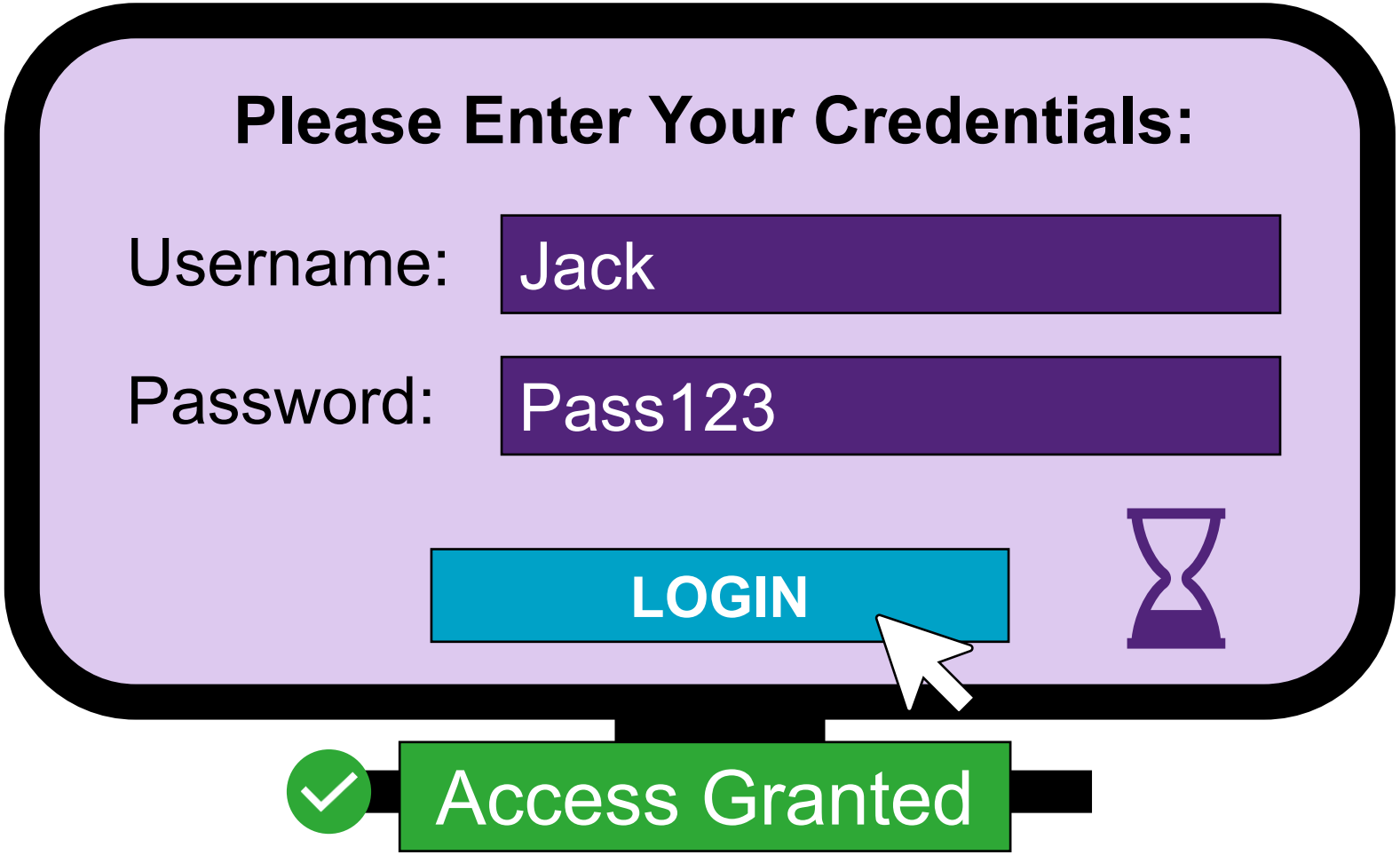
Id	FullName	Username	Password
1	Jack Smith	Jack	Pass123

Result Returned? Yes

Access Granted

SQL Manipulation Example

Regular behavior



SQL Manipulation Example

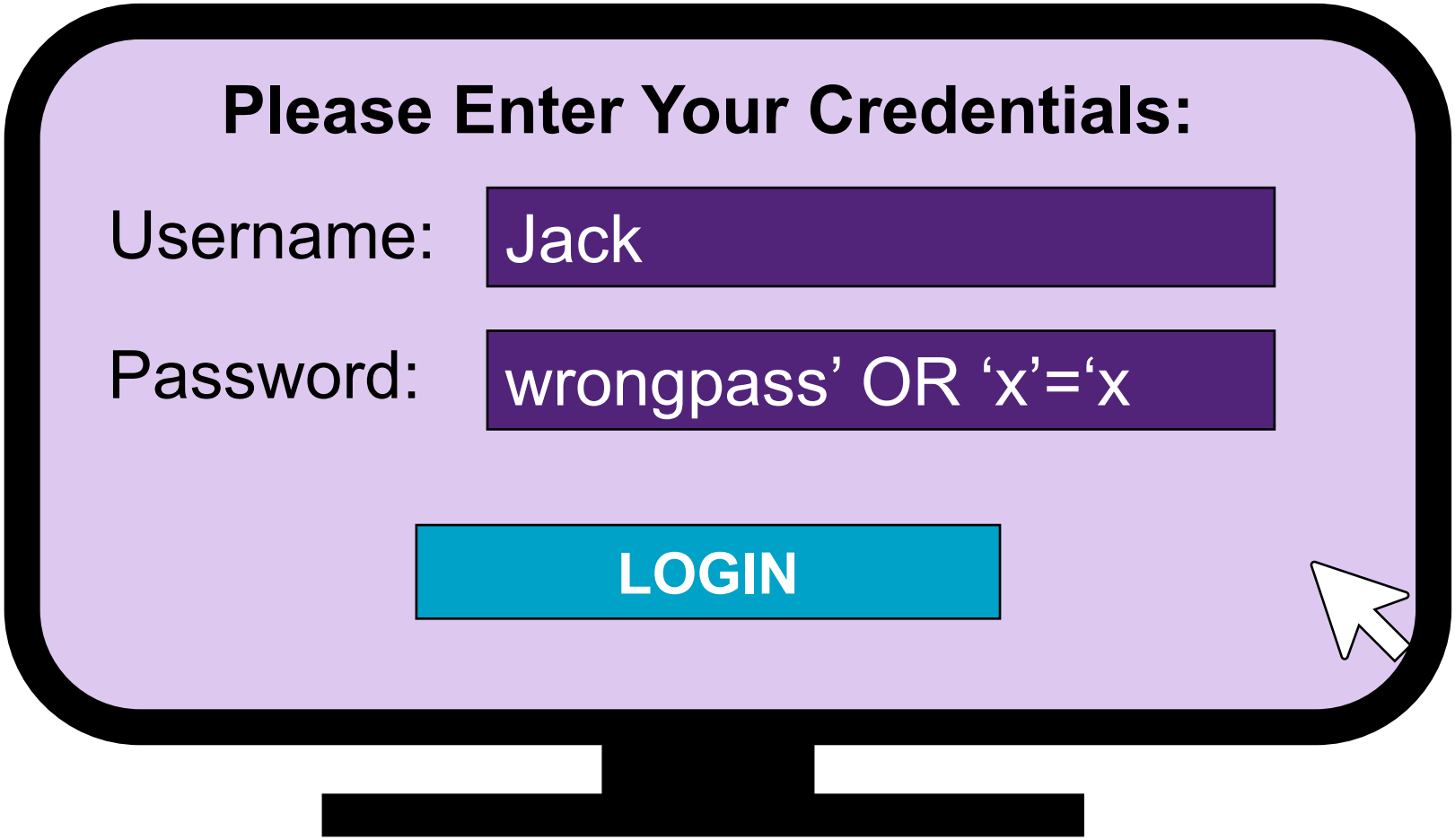
Key Points:

- The client's input is directly used in the SQL query
- The SQL query needs to return a tuple in the User table for the client to be authenticated

Let's see now how a Hacker that knows the username of a user can take advantage of these two points to gain access to a system 😎

SQL Manipulation Example

Hacker behaviour



SQL Manipulation Example

Hacker behaviour

DBMS

```
SELECT *
FROM Users
WHERE username = ('Jack' AND password = 'wrongpass') OR 'x'='x')
```

Id	FullName	Username	Password
1	Jack Smith	Jack	Pass123

Result Returned? Yes

Access Granted

SQL Manipulation Example

Hacker behavior

Please Enter Your Credentials:

Username: Jack

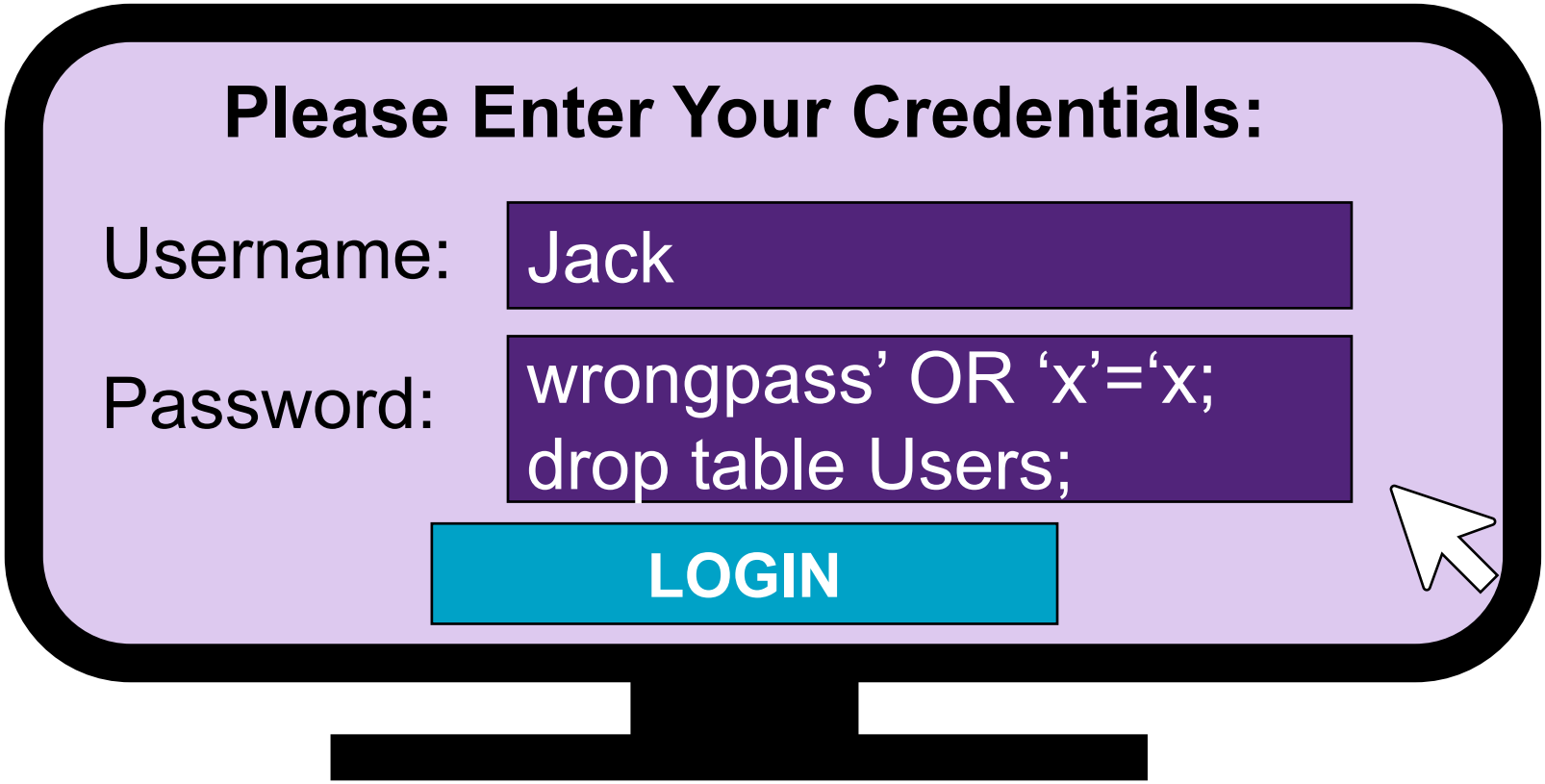
Password: wrongpass’ OR ‘x’=‘x

LOGIN

Access Granted

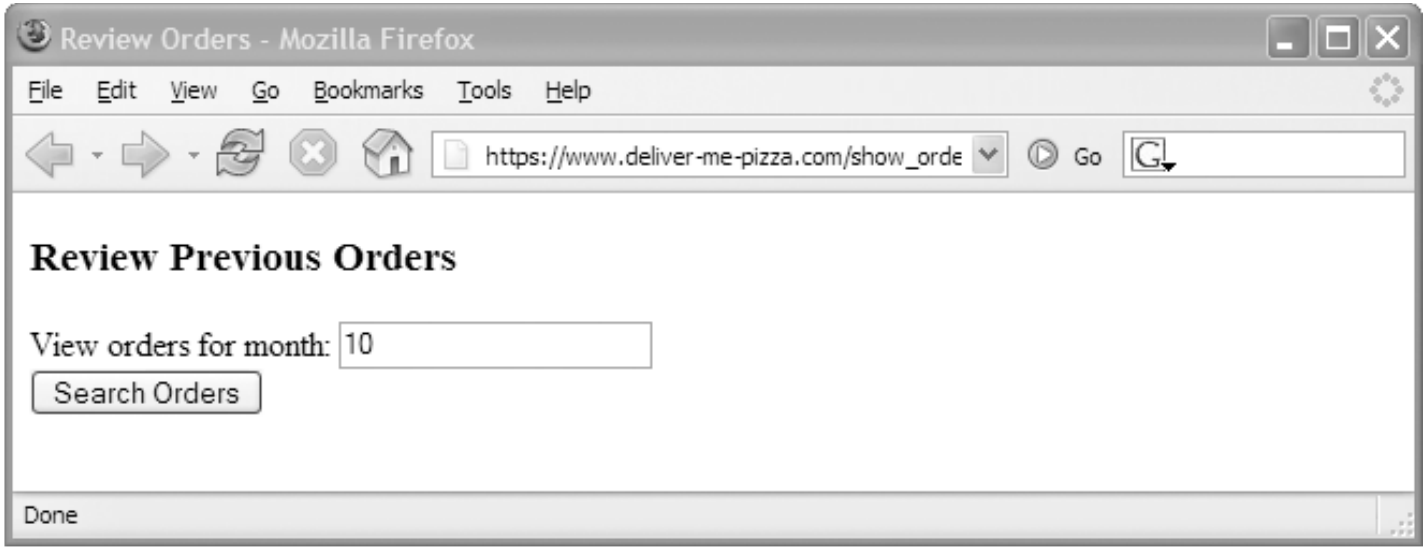
Code injection

Code injection: This type of attack attempts to add additional SQL statements or commands to the existing SQL statement by exploiting a computer bug, which is caused by processing invalid data.



Example - 1

Consider a browser form, e.g.:



When the user enters input (10) the following http request is generated:

https://www.pizza.com/show_orders?month=10

Example - 2

After receiving the http request, an SQL query is generated as follows:

```
sql_query
      = "SELECT pizza, topping, quantity, order_day "
          + "FROM orders "
          + "WHERE userid=" + session.getCurrentUserId()
          + " AND order_month= "
          + request.getParameter("month");
```

For example, this can pass a query to the database as following for *userid=4123* in a normal situation:

```
SELECT pizza, topping, quantity, order_day
FROM orders
WHERE userid=4123
AND order_month=10
```

Example - 3

What if a malicious user makes a modified http request:

```
0 OR 1=1
```

Parameters transferred in URL-encoded form, where meta-characters are encoded in ASCII (space : %20, = : %3D)

```
https://www.pizza.com/show_orders?month=0%20OR%201%3D1
```

And finally resulting:

```
SELECT pizza, topping, quantity, order_day
FROM orders
WHERE userid=4123
AND order_month= 0 OR 1=1
```

Example - 4

Since AND takes precedence over OR the SQL query:

```
SELECT pizza, topping, quantity, order_day
FROM orders
WHERE (userid=4123
AND order_month=0) OR 1=1
```

The above query always evaluates to TRUE and the attacker gets every entry in the *order* table

Comment Indicator in SQL

Double-dash sequence -- is a comment indicator in SQL

The rest of the query is interpreted as a comment and ignored

```
SELECT * FROM users WHERE username= ' '  OR 1=1-- '
```

SQL Injection Using Union

Send a request that generates an SQL query like the following:

```
SELECT pizza, topping, quantity, order_day
FROM orders
WHERE userid=4123
AND order_month=0 AND 1=0
UNION SELECT cardholder, number, exp_month, exp_year
FROM creditcards
```

Attacker gets the entire credit-card table.

False condition which results empty table for the first *select* statement.

SQL Injection Using Multiple Queries

SQL queries be separated to multiple statements using ';'

A request that generates an SQL query like the following:

```
SELECT pizza, topping, quantity, order_day
FROM orders
WHERE userid=4123
AND order_month=0 ;
DROP TABLE creditcards
```

Will delete credit-card table after retrieving data from the *orders* table

SQL Injection: Detection & Bypassing -1

You must log in to proceed
Please enter your name and password

name:

password:

Submit Query

```
SELECT user FROM employees
WHERE user='admin' AND
password = 'password'
```

Valid query

```
SELECT user FROM employees
WHERE user='admin' AND
password=''
```

Check to see if password field is injectable

Unclosed quotation mark after the character string ''.
Incorrect syntax near ''.

Description: An unhandled exception occurred during the execution of the current web request. Please review the stack trace for more information about the error and where it originated in the code.

Exception Details: System.Data.SqlClient.SqlException: Unclosed quotation mark after the character string ''.
Incorrect syntax near ''.

SQL Injection: Detection & Bypassing -2

You must log in to proceed
Please enter your name and password

name:

password:

Submit Query

```
Password: ' or '1'='1
```

```
SELECT user FROM employees
WHERE user='admin' AND
password='' or '1'='1'
```

Query is valid and return one row if “user=admin” exists.

Discovering the Database and Table Name

Discovering database name

```
SELECT pizza, topping, quantity, order_dayFROM
orders WHERE userid=4123
UNION SELECT DATABASE(), NULL, NULL, NULL;
```

Discovering table name

```
SELECT pizza, topping, quantity, order_day FROM
orders WHERE userid=123
UNION SELECT table_name, NULL, NULL, NULL FROM
information_schema.tables WHERE table_schema =
DATABASE()
```

- UNION SELECT: Combines results from the attacker's and original query.
- table_name: Retrieves table names.
- information_schema.tables: Stores metadata for all tables.
- WHERE table_schema = DATABASE(): Limits results to tables in the current database.

pizza	topping	quantity	order_day
Margherita	Basil	1	2023-10-22
Pepperoni	Pepperoni	2	2023-10-21
orders	NULL	NULL	NULL
users	NULL	NULL	NULL
creditcards	NULL	NULL	NULL

Determining the Number of Columns

Injecting a series of ORDER BY clauses and incrementing the specified column index until an error occurs

```
SELECT username, email from users where username='x' ORDER BY 1;
SELECT username, email from users where username='x' ORDER BY 2;
SELECT username, email from users where username='x' ORDER BY 3;
```

Suppose ORDER BY 4 returns an error, then, we know the number of columns of users table is 3.

Finding a Value

Injecting a query to determine the first character of the database name and keep extracting characters one by one.

```
SELECT 'A' =(SELECT MID(Name,1,1) FROM Users);
```

The result of above query is one if the first character of field Name from table Users is 'A'; if not the result of query is 0

If we repeat this for each character, we can discover the whole name

An improved version is to perform a binary search

```
SELECT (SELECT ORD(MID(Name,1,1))FROM Users)<=77;
```

- MID: substring extraction command
- ORD: returns the ASCII code
- 77: ASCII code for character 'M'

Risks Associated with SQL Injection

Database fingerprinting: Attacker determines the type of database being used.

Denial of service: Attacker denies service to valid users.

Bypassing authentication: attacker gains access to database without valid authorization.

Identifying injectable parameters: attacker determines information about backend structure

Executing remote commands: attacker executes harmful commands remotely.

Performing privilege escalation: attacker upgrades their access level

Protection Techniques

SQL injection attacks can occur when SQL statements are used in programs

Bind variables (using parameterized statements)

- Protects against injection attacks
- Improves performance

Filtering input (input validation)

- Remove escape characters from input strings
- Escape characters can be used to inject manipulation attacks

Function security

- Standard and custom functions should be restricted

Review

Do you know …

- What are the threats to database security?
- What control measures can be taken to provide security of data in relational databases?
- What is SQL injection and how can it be used to compromise database security?

Reading

- Chapters 30 in Elmasri & Navathe

Final Exam

- ✓ April 22, 3:30 PM – 6:30 PM | SRYE 1002
- ✓ A combination of multiple-choice questions and short answer questions
- ✓ Similar to in-class exercises, assignments, practicals and tutorials
- ✓ Closed book and covers all modules

- Modules 0, 1 – 10 points
- Module 2 – 20 points (but also relies on module 1)
- Module 3 – 30 points
- Module 4 – 25 points
- Module 5 – 15 points

Revision Checklist for Module 1

Terminology

1. Entity (Weak entity)
2. Relationship (Degree, Recursive)
3. Attribute (Key and Partial Key, Composite, Multivalued, Derived)
4. Constraints (Cardinality Ratio, Participation Constraints)
5. Extended ER (Specialization, Generalization)

Basic Concepts

1. Conceptual Modeling
2. ER Diagram
3. Design Choices

Exam Questions (10 points)

- Functionality of RDMBs
- ER Comprehension (answering questions about ER)

Revision Checklist for Module 2

Terminology

1. Relations (Is a Set, Rows and Columns)
2. Domains (Atomicity, Data types)
3. Attributes (Degree of a relation. Prime or Key attribute)
4. Tuples
5. Key (Super key, Minimal key, Candidate key, Foreign key)
6. Mapping

Basic Concepts

1. Integrity Constraints (Domain, Key, Entity, Referential)
2. Constraint violations, Constraint and operations, inconsistent database state
3. Step by stem process for mapping

Exam Questions (20 points)

- Identifying keys and constraint violations
- Doing a full mapping of an ER diagram to relations

Revision Checklist for Module 3

Terminology

1. Declarative vs. Procedural
2. DDL statements (CREATE TABLE, ALTER TABLE, DROP TABLE)
3. DML statements (INSERT, DELETE, UPDATE, SELECT)

Basic Concepts

1. Selection, Projection, Sorting
2. Aggregation (COUNT, SUM, AVG, MIN, MAX) and Grouping (GROUP BY)
3. Conditions on grouping and aggregation (Having)
4. Multiple relation queries (Join, Nesting)
5. When to join and when to nest?
6. Correlated and non-correlated queries
7. Sub-query operation (IN, comparison with (or without) ANY/ALL, EXISTS)

Exam Questions (30 points)

- Writing SQL queries

Revision Checklist for Module 4

Terminology

1. Anomalies
2. Functional dependencies
3. Normal Forms

Basic Concepts

1. How to determine and infer FDs
2. Closure computation
3. Definition of Normal Forms
4. Normalization is a process
5. Algorithms for BCNF and 3NF decomposition

Exam Questions (25 points)

- Identify keys and closures
- Identify highest normal form
- Decompose into BCNF, put FDs in minimal cover and synthesis into 3NF

Revision Checklist for Module 5

Terminology

1. Privacy vs. Security
2. Sensitive Data
3. Database Audit
4. Access Control

Basic Concepts

1. Threats to Database Security
2. Database Control Measures
3. SQL Injection

Exam Questions (15 points)

- Determine privileges
- Show understanding of different access control mechanisms
- Show understanding of SQL injection